

Lab 0 - C23 Survival Guide

Modern C23 toolchain, memory, pointers, structs, and program organization

Student	_____
Date	_____
Repository / folder	_____

Purpose This is not a full course in C. It is a survival lab: learn enough modern C23 to compile, inspect, debug, and reason about the programs you will use to study data structures.

Learning outcomes

- Compile a C23 program with strict warnings and sanitizers enabled.
- Use loops, conditionals, functions, arrays, and pointers correctly.
- Explain the pointer + length contract used when arrays are passed to functions.
- Explain the contract of a null-terminated C string and safely use `strlen()` and `strcmp()` on valid strings.
- Separate declarations and implementations using `.h` and `.c` files.
- Create structs, use `typedef`, and access a struct through a pointer.
- Allocate and release dynamic memory, check allocation failure, and use C23 `nullptr` for null pointers.
- Explain what a self-referential struct stores and why it matters for linked data structures.
- Build a multi-file C23 program with a simple Makefile.

The rule for this course

C23, warning-clean, sanitizer-clean. A program that “runs” is not automatically a correct program. During development, your code must compile with the required diagnostics and run without sanitizer errors.

Throughout this lab, use the following development command unless an exercise tells you otherwise:

```
gcc -std=c23 -Wall -Wextra -Wpedantic -Wconversion -Wshadow \
-Wformat=2 -Wvla -Werror -g3 -O1 -fno-omit-frame-pointer \
-fsanitize=address,undefined hello.c -o hello
```

If you use Clang, replace `gcc` with `clang`. Do not remove a warning flag merely to make a diagnostic disappear; understand and correct the code.

Modern C23 null-pointer convention. Write `nullptr` whenever you mean “no pointer.” You may encounter `NULL`, integer `0`, or `(void *)0` in older C code, but do not use those spellings for null pointers in new CMPS2131 code.

Part 1 - First clean C23 build

Create a folder named `lab1` and, inside it, create `hello.c`.

```
#include <stdio.h>

int main(void)
{
    puts("Hello, C23.");
    return 0;
}
```

1. Compile the program using the required development command.

2. Run it with `./hello`.
3. Change `main(void)` to `main()`, compile again, and observe that it is still a no-parameter function in C23. Restore `main(void)`; this course style uses the explicit form to make the contract obvious.
4. Introduce a simple warning (for example, declare a local variable and do not use it). Observe what `-Werror` does. Then fix the program.

☑ My program compiles with zero warnings and runs successfully.

Checkpoint A - Explain before moving on

- What does `-std=c23` control? It basically tells the compiler the language to use.
- What is the difference between a compiler warning and a compiler error? A warning could refer to some code that might be wrong but an error means that somewhere in the program some code is wrong
- Why might a course deliberately convert warnings into errors? To further improve problem solving skills.
- What do `AddressSanitizer` and `UndefinedBehaviorSanitizer` help you detect? `AddressSanitizer` helps detect memory errors and `undefinedBehaviorSanitizer` helps detect undefined behavior within your program.

Part 2 - Loops, conditionals, and functions

Create `basics.c`. Complete the following requirements without using global variables.

5. Write a loop that prints the integers 1 through 10.
6. Inside the loop, use a conditional to print whether each number is even or odd.
7. Create `int square(int value)` and use it to print the square of each number.
8. Create `bool is_even(int value)` and use the function in your conditional. In C23, `bool` is a language keyword; do not include `<stdbool.h>` merely to obtain `bool`.

Contract question What must the caller provide to `square()`? What does `square()` promise to return? Begin thinking of every function as a small contract between caller and implementation.

The caller needs to provide a `int` to `square()`. `square()` promises to return the square of that value.

☑ `basics.c` compiles warning-clean and sanitizer-clean.

Part 3 - Arrays are data; pointers do not carry lengths

Create `arrays.c`. Start with this array:

```
int values[] = {4, 7, 1, 9, 3};
size_t count = sizeof values / sizeof values[0];
```

Write the function below and call it from `main`:

```
int sum_array(const int *values, size_t count);
```

9. Use `size_t` for indexes and counts.
10. Do not hard-code the number 5 inside `sum_array()`.
11. Print the count, each element, and the final sum.
12. Change the array to contain seven elements. Your function must continue to work without modification.

Mental model When an array is passed to a function, the function receives a pointer to elements - not the array length. The caller must provide the length separately. The pointer + length pair is part of the function contract.

Checkpoint B - Reason about memory

- Why is `size_t` a better type than `int` for an array length? Its better because it cant be negative and is made for specific sizes and counts

- Can `sum_array()` discover how many elements are available from values alone? No because values is just a pointer not holding any information about the elements inside the array
- What does `const` promise about the elements as viewed through values? It promises that `sum_array` will not change any elements in the array.
- What could happen if `count` is larger than the number of valid elements? It would just keep running and finish and fill the values with random numbers.

Part 4 - Pointers: addresses and indirection

Create `pointers.c`. Use the following starting point:

```
int score = 10;
int *score_ptr = &score;
```

13. Print `score`.
14. Print the address stored in `score_ptr` using `%p` and cast the pointer to `(void *)`.
15. Print the value reached through `score_ptr`.
16. Change `score` to 25 by writing through `score_ptr`.
17. Print `score` again and explain why it changed.

`score_ptr` points to `score`, so changing the value through the pointer also changes `score`.

Required form for printing a pointer:

```
printf("%p\n", (void *)score_ptr);
```

Do not memorize “pointer syntax” in isolation. A pointer is a value that stores an address. Dereferencing follows that address to an object. The syntax makes more sense when you keep the memory model in view.

Part 5 - C strings are a memory convention

Create `strings.c`. This exercise is intentionally small. You are learning the representation and contract, not a collection of legacy string-manipulation functions.

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char word[] = "Belize";

    printf("length = %zu\n", strlen(word));

    if (strcmp(word, "Belize") == 0) {
        puts("The strings are equal.");
    }

    return 0;
}
```

18. Compile and run the program.
19. Draw the bytes conceptually as `B e l i z e \0`. 0, 1, 2, 3, 4, 5, 6
20. Explain how `strlen()` knows where to stop even though you did not pass a length. Because when it reaches the null terminator it knows to stop
21. Change the comparison string and observe `strcmp()`. Doesn't print out text, only prints out the length

Security rule `strlen()` and `strcmp()` are appropriate only when their arguments are valid null-terminated C strings. They do not make an arbitrary byte buffer safe. In this course, do not treat `strcpy()`, `strcat()`, `sprintf()`, or similar legacy copying/concatenation patterns as default string-handling techniques.

Do NOT deliberately call `strlen()` or `strcmp()` on an array that lacks a null terminator. That violates the function contract and invokes undefined behavior.

Part 6 - Separate interface from implementation

Create three files: `stats.h`, `stats.c`, and `main.c`. The header describes what another translation unit may call; the `.c` file contains the implementation.

`stats.h`

```
#ifndef STATS_H
#define STATS_H

#include <stddef.h>

int sum_array(const int *values, size_t count);

#endif
```

`stats.c`

```
#include "stats.h"

int sum_array(const int *values, size_t count)
{
    int total = 0;

    for (size_t i = 0; i < count; ++i) {
        total += values[i];
    }

    return total;
}
```

Write `main.c` yourself. It must include `stats.h`, create an array, compute its length, call `sum_array()`, and print the result.

Compile all source files together:

```
gcc -std=c23 -Wall -Wextra -Wpedantic -Wconversion -Wshadow \
    -Wformat=2 -Wvla -Werror -g3 -O1 -fno-omit-frame-pointer \
    -fsanitize=address,undefined main.c stats.c -o stats_demo
```

Checkpoint C - Interface vs implementation

- Why does `main.c` include `stats.h` instead of `stats.c`? It is to connect files to get the information
- What belongs in a header file? Function Declarations
- What problem do the include guards solve? It stops a header file from being in the same file more than once
- Why is separate compilation useful once programs become larger? Its more useful because code is better organized and easier to manage

Part 7 - Structs and typedef

Create `record.h` and define the following type:

```

#ifndef RECORD_H
#define RECORD_H

typedef struct {
    int id;
    double value;
} Record;

#endif

```

In main.c:

22. Create one Record object using an initializer.
23. Print its fields using the . operator.
24. Create Record *record_ptr that points to the object.
25. Print the same fields through the pointer using ->.
26. Change value through the pointer and verify that the original object changed.

Interpretation record.value means “select a field from this object.” record_ptr->value means “follow this pointer to an object, then select the field.”

Part 8 - Dynamic memory and lifetime

Now create a Record whose lifetime is controlled explicitly rather than by the current block.

```

#include <stdio.h>
#include <stdlib.h>
#include "record.h"

int main(void)
{
    Record *record = malloc(sizeof *record);

    if (record == nullptr) {
        fputs("memory allocation failed\n", stderr);
        return EXIT_FAILURE;
    }

    record->id = 1;
    record->value = 42.5;

    printf("id=%d value=%.1f\n", record->id, record->value);

    free(record);

    return EXIT_SUCCESS;
}

```

27. Compile and run this program with the sanitizers enabled.
28. Explain why malloc() must be checked before dereferencing record. It has to be cleared or else malloc() can fail and return nullptr
29. Explain why sizeof *record is preferable here to repeating sizeof(Record). Sizeof *record stays correct even if you change the type of record
30. Temporarily comment out free(record), run the program, and observe whether your AddressSanitizer configuration reports the leak. Restore free(record) afterward. If your local sanitizer does not report leaks, record that observation rather than changing the code. After commenting out free(record), AddressSanitizer did not report a memory leak.
31. Explain what becomes invalid after free(record). The pointer record becomes invalid

Security rule Never dereference a pointer returned from an allocation function before checking for failure. Never use an object after its lifetime has ended. Every successful owned allocation needs a clear ownership and release plan. Assigning one pointer variable to nullptr does not make other aliases safe after free().

Part 9 - The bridge to linked data structures

Add this type to a new file named node.h:

```
#ifndef NODE_H
#define NODE_H

typedef struct Node {
    int value;
    struct Node *next;
} Node;

#endif
```

Do not build a linked list yet. Create only one dynamically allocated Node. Set value to 100 and next to nullptr, print value, then release the node.

Stop and reason The next field does not contain another Node. It contains an address at which another Node could exist. That single idea is the bridge from ordinary records to linked structures.

- Why does the struct need the tag Node in struct Node *next? The tag is needed so the compiler knows the type of next
- What does next == nullptr communicate? It means there is no next node
- If next eventually points to another Node, where might that second Node live? It would live in dynamically allocated memory
- Who should be responsible for freeing dynamically allocated nodes? The programmer must free every node they allocate

Part 10 - Build the program with Make

Your final folder should contain at least:

```
lab1/
├── Makefile
├── main.c
├── stats.c
├── stats.h
├── record.h
└── node.h
```

Create this Makefile. Recipe lines must begin with a TAB character, not spaces.

```

CC = gcc
CFLAGS = -std=c23 -Wall -Wextra -Wpedantic -Wconversion -Wshadow -Wformat=2 -Wvla -Werror -g3 -O1
-fno-omit-frame-pointer
SANITIZERS = -fsanitize=address,undefined
TARGET = lab1
OBJECTS = main.o stats.o

.PHONY: all run clean

all: $(TARGET)

$(TARGET): $(OBJECTS)
    $(CC) $(OBJECTS) $(SANITIZERS) -o $(TARGET)

main.o: main.c stats.h record.h node.h
    $(CC) $(CFLAGS) $(SANITIZERS) -c main.c -o main.o

stats.o: stats.c stats.h
    $(CC) $(CFLAGS) $(SANITIZERS) -c stats.c -o stats.o

run: $(TARGET)
    ./$(TARGET)

clean:
    rm -f $(TARGET) $(OBJECTS)

```

If you use Clang, run `make CC=clang` instead of editing the Makefile.

32. Run `make clean`.
33. Run `make`.
34. Run `make run`.
35. Fix every compiler or sanitizer diagnostic. Do not submit a warning-producing build.

Final integration task

Modify `main.c` so one execution demonstrates the concepts from the lab without becoming a data-structure implementation. Your final program must:

- ☐ Create an ordinary integer array and call `sum_array()`.
- ☐ Create one `Record` object and access it through a pointer.
- ☐ Allocate one `Record` dynamically, check the result, use it, and free it.
- ☐ Allocate one `Node` dynamically, set `next` to `nullptr`, use it, and free it.
- ☐ Return `EXIT_SUCCESS` on success.
- ☐ Compile through the Makefile with zero warnings.
- ☐ Run without `AddressSanitizer` or `UndefinedBehaviorSanitizer` errors.

What not to use in this lab

Unless your instructor explicitly asks for them, do not introduce additional complexity. In particular, do not use:

- `strcpy()`, `strcat()`, `sprintf()`, `gets()`, or “safe because it has `n` in the name” substitutions without understanding their contracts;
- legacy null-pointer spellings such as `NULL`, integer `0`, or `(void *)0` in new code; use `nullptr`;
- variable-length arrays;
- pointer arithmetic beyond ordinary array indexing;
- macros beyond `include guards`;
- global mutable state;
- function pointers, unions, bit fields, or advanced preprocessor techniques.

Submission

Submit the complete lab1 folder or repository link according to your instructor's normal submission method. Before submitting, verify all of the following:

- ☐ make clean && make completes successfully.
- ☐ The build produces no warnings.
- ☐ make run completes without sanitizer errors.
- ☐ All dynamically allocated memory in the final program has a clear matching free().
- ☐ All null pointers in new code are written using nullptr.
- ☐ No required compiler warning or sanitizer option was removed to hide a problem.
- ☐ Your source is organized into appropriate .h and .c files.
- ☐ You can verbally explain every pointer used in your final program.

Exit ticket - answer in your own words

1. What information does a pointer contain?

A pointer contains a memory address

2. Why does a function receiving an array usually also need a length?

Because the function only gets a pointer but not the size of the array.

3. What makes a char array a valid C string?

It must end with a null terminator

4. What is the difference between a struct object and a pointer to a struct object?

The struct is the real object and the pointer just holds its address.

5. What must happen after malloc() before you dereference the returned pointer?

Check that malloc did not return a nullptr.

6. What does free() change about the lifetime of an allocated object?

It ends the object's lifetime and then you can no longer use that memory.

7. In a self-referential Node, what exactly is stored in next?

The address of another Node or a nullptr

8. Why is "it printed the right answer once" not enough evidence that a C program is correct?

The program might still have hidden bugs or undefined behavior.